National University of Computer & Emerging Sciences, Karachi
Spring 2021 CS-Department
Final Examination Solution
08th June 2021, 12:30 pm – 02:00 pm

Course Code: CL 220	**Course Name: Operating Systems Lab**
Instructor Name / Names: Tania Iram, Anaum Hamid, Safia, Rabia, Ali Fatmi	
Student Roll No:	**Section No:**
PaperAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA	

Instructions:

- Return the question paper.
- Read each question completely before answering it. There are **5 questions and 4 pages**
- In case of any ambiguity, you may make assumption. But your assumption should not contradict any statement in the question paper.

Time: 90 minutes. **Max Marks:** 50 points

Q1. Signal Handling Marks 5

Complete the grey areas of program that receives the interrupts signal and shows the counter number on each interrupt signal. If program receives the stop signal, it shows the total number of signals it has received and terminates itself. While the program is receiving no signals, it keeps printing the message "Program is sleeping" after each second. Grey area can have two or more lines.

```
#include <signal.h>
#include <stdio.h>
#include <wait.h>
#include <stdlib.h>
#include <unistd.h>

static int count=1;
 void signalHandler( int signum) {
printf("Interrupt signal (%d) received.\n",count);
        count++;

 }

 void signalHandler1( int signum) {
printf("Terminatingthe program with %d",count);
        exit (signum);

 }
int main () {
   signal(SIGINT, signalHandler);
   signal(SIGTSTP, signalHandler1);
  while(1) {
    printf( "program is sleeping....\n");
    sleep(1);
  }
  return 0;
```

}

Q2. Shell scripting, Process Management and Communications **Marks 10**

a) Write a program 'renaming.sh' to rename file1.jpg as 'as my_file.jpg'
b) write a c program to send the name of the shell script in part a from parent process to child process using pipes. child will then execute that shell script and exit.

C program

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
int main(int argc, char *argv[])
{

int pipefd[2];
        int pid;
        char buffer[30];
        pipe(pipefd);
        pid = fork();

        if(pid > 0) {
                fflush(stdin);
                printf("unamed_pipe [INFO]
Parent Process\n");

        write(pipefd[1],"./renaming.sh",30);
        }
        else if(pid == 0) {
                sleep(5);
                fflush(stdin);
                printf("unamed_pipe [INFO]
Child Process\n");
                read(pipefd[0], buffer,
sizeof(buffer));
                write(1,buffer,
sizeof(buffer));
                printf("\n");
execlp(buffer,buffer,NULL);
        }
        else {
                printf("unamed_pipe [ERROR]
Error in creating child process\n");
        }
        if(pid > 0) wait();
        return 0;
return 0;
}
```

Renaming.sh

```
rename renameing.sh file1.jpg
```

Q3: Multithreaded Programming using Pthreads **Marks 10**

Complete the code of a simple multithreaded program in C that create multiple threads by default attributes on Linux environment and each thread will print the table till 10 with its Id number. Perform necessary error checking where needed. In given Code Grey area. main has 1 and in thread has more than 2 lines missing.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#define N 4
void *child_thread(void *arg)
{
 printf("This is thread #%ld\n", (long)arg);
        int i;
        for (i = 1; i <= 10; ++i) {
        printf("%ld * %d = %ld\n", (long)arg, i, (long)arg * i);
    }
        pthread_exit(NULL);



}
int main()
{       pthread_t my_thread[N];
       long id;
       for(id = 1; id <= N; id++) {
              int value =  pthread_create(&my_thread[id], NULL, &child_thread, (void*)id);
              if(value != 0) {
                     printf("Error: pthread_create() failed\n");
                     exit(EXIT_FAILURE);              }
       }              pthread_exit(NULL);
}
```

Q4: Multithreaded Programming using OpenMp **Marks 15**

In stock market the shares of companies go up and down during a day. Write an OpenMp program for stock market shares update according to the given requirement:

- You need to initialize two arrays with random values.
- Array 'a[8]' will hold the opening price of stock for 8 companies.
- Array d[8] will be initialized with random values between one to ten. (use rand() function).
- Four OpenMp threads will update the values of two companies, once only, in each iteration after checking the contents of d[i] (criteria is increase the share value by d[i] value if value greater than 5 otherwise decrease by the value of d[i]).
- Display which thread update the value of which company
- Display the closing price of shares of each

```
#include <stdlib.h>   //malloc and free
#include <stdio.h>    //printf
#include <omp.h>      //OpenMP

// Very small values for this simple illustrative example
#define ARRAY_SIZE 8     //Size of arrays showing number of companies.
#define NUM_THREADS 4    //Number of threads to use for vector addition or subtraction.

int main (int argc, char *argv[])
{
    // elements of arrays a and b will be added
    // and placed in array c
    int * a;
    int * c;
      int * d;
      int n = ARRAY_SIZE;              // number of array elements
```

```
        int n_per_thread;                   // elements per thread
        int total_threads = NUM_THREADS;    // number of threads to use
        int i;       // loop index

          // allocate spce for the arrays
          a = (int *) malloc(sizeof(int)*n);
        c = (int *) malloc(sizeof(int)*n);
        d = (int *) malloc(sizeof(int)*n);
          // initialize arrays a and b with consecutive integer values
        // as a simple example
          for(i=0; i<n; i++) {
              a[i] = i+100;
              }

        // Additional work to set the number of threads.
        // We hard-code to 4 for illustration purposes only.
        omp_set_num_threads(total_threads);

        // determine how many elements each process will work on
        n_per_thread = n/total_threads;

          // Compute the vector addition
        // Here is where the 4 threads are specifically 'forked' to
        // execute in parallel. This is directed by the pragma and
        // thread forking is compiled into the resulting exacutable.
        // Here we use a 'static schedule' so each thread works on
        // a 2-element chunk of the original 8-element arrays.
        #pragma omp parallel for shared(a, c) private(i)
          for(i=0; i<n; i++) {
                  d[i]=rand()%10;
                          if (d[i]>5)
                          c[i] = a[i]+d[i];
                          else
                          c[i]= a[i]-d[i];
                  // Which thread am I? Show who works on what company for this samll example
                  printf("Thread %d update shares of company%d\n", omp_get_thread_num(), i);
          }
        printf("====Final values of shares in stock market====\n");
        printf("i\ta[i]\t+or-\td[i]=\tc[i]\n");
          for(i=0; i<n; i++) {
                  printf("%d\t%d\t\t%d\t%d\n", i, a[i],d[i], c[i]);
          }

          // clean up memory
          free(a);  free(c);

        return 0;
    }
```

Q3: Process Synchronization **Marks 10**

Complete the code for a synchronize patient checkups mechanism in the hospital using semaphore where there are 10 patients and 2 doctors, each needs to wait to get his appointment token/number. During each task patient feel sleepy so they sleep, weighting area takes 5 seconds sleep, and the doctor will usually take 10 sec for checkup, so patient need 10 seconds sleep. Main function is given Write the handler function code only.

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
#include<string.h>
#include<unistd.h>
sem_t appointment, waiting;

void handler(void* ptr) {

int i=*(int*)ptr;


sem_wait(&appointment);
printf(" patient %d waiting for token\n", i+1);
sleep(5);
sem_post(&appointment);
printf(" patient %d get token\n", i+1);

sem_wait(&waiting);
printf(" patient %d waiting for trun\n", i+1);
sleep(10);
sem_post(&waiting);
printf("patient %d getting checkup finished\n", i+1);




}
int main() {
int i,a[10];
pthread_t patient[10];
sem_init(&appointment,0,1);
sem_init(&waiting,0,2);
for(i=0;i<10;i++) {
a[i]=i;
pthread_create(&patient[i],0, (void*)handler, (void*)&a[i]);
}
for(i=0;i<10;i++) {
pthread_join(patient[i],NULL);

}
printf("Done");
sem_destory(&appointment);
sem_destory(&waiting);
return 0;
}
```